Harnessing Agent Skills: Architectural Patterns and a Reference Architecture for Skill-Mediated LLM Agents

Boming Xia[a,b,c], Liming Zhu[c,d], Zhenchang Xing[c], Qinghua Lu[c,d,a], Dino Sejdinovic[a,b] and Xiwei Xu[c,d]

[a]Responsible AI Research (RAIR) Centre, Adelaide, SA, Australia
[b]Adelaide University, Adelaide, SA, Australia
[c]CSIRO, Sydney, NSW, Australia
[d]University of New South Wales, Sydney, NSW, Australia

arXiv:2606.20631v2 [cs.AI] 2 Nov 2026

ARTICLE INFO

Keywords:
Skill
Agent skill
Harness engineering
Agent harness
Pattern
Reference architecture
Agentic
Agentic engineering

ABSTRACT

CONTEXT: Agent skills package reusable behavioural guidance and resources that large language model agents load or apply during a run. Within the agent harness, agent skill harnessing comprises connected architectural responsibilities that govern which skills enter a run, which actions they can cause, and how their use is recorded and informs revision. OBJECTIVES: We identify architectural patterns for agent skill harnessing and organise their responsibilities into a reference architecture that makes these connections explicit. METHODS: A multivocal literature review covered 36 practitioner systems and 51 academic papers; we consolidated practices into patterns and derived the reference architecture from patterns, source evidence, and architectural reasoning. We evaluated the reference architecture by mapping eight systems and applying it in Deep Agents and Hermes Agent. RESULTS: Ten patterns cover skill supply and activation, skill declarations and execution authority, verification and recording of skill use, and run evidence feeding repair and governed revision. The reference architecture organises their responsibilities into four layers (Supply Chain, Mediation, Execution Control, Evidence & Feedback) and two cross-cutting substrates, separating permission determination from enforcement and routing persistent revisions through acceptance. Mapping eight systems showed the first three layers broadly realised and 14 of 15 partial or unestablished ratings in Evidence & Feedback. Applying the architecture in Deep Agents and Hermes Agent turned requirements into checks that exposed a shell route outside the file-tool rules and a preview–apply mismatch. CONCLUSION: The patterns and reference architecture support comparing implementations and turning requirements into responsibility allocations and checks.

1. Introduction

Agentic AI systems allocate behavioural capability across prompts, retrieval, memory, workflows, callable tools, and externalised behavioural artefacts, alongside the capabilities encoded in model weights [1, 2]. *Agent skills* are one such artefact form: reusable units of behavioural guidance or intervention logic stored outside model weights, together with associated metadata and resources, for use by a large language model (LLM) agent. They include instruction-based packages and programmatic forms that intervene in the agent loop [3, 4, 5]. Externalising these resources allows them to be inspected and revised independently of the model. It also places decisions about their availability, selection, use, and evolution in the surrounding software system. Because these decisions span components and affect runtime control, trust boundaries, and traceability, their interaction is an architectural concern.

The central issue is how a reusable skill artefact participates in a particular agent run. Participation depends on the task, available context, mechanisms of the host (the agent application or platform), and applicable policies. We call this run-specific participation *skill-in-use*, and the associated observable actions and outputs *skill-mediated behaviour* (Figure 1). Skill participation also interacts with the permissions governing execution. The Agent Skills specification includes experimental `allowed-tools` pre-approval metadata, with host-dependent support, and GitHub Copilot CLI documents pre-approval of listed tools [6, 7]. What an agent may do under a skill therefore depends on the applicable permissions as well as on the skill's instructions.

These participation and authorisation decisions connect to admission, execution control, evidence capture, and persistent change. We use *agent skill harnessing* for the architectural responsibilities and mechanisms that govern skill supply, runtime participation, and executable effects, and link them to evidence and revision. It sits within the broader harness that coordinates context, tools, execution, and observation [2]. Section 2 develops the scope and definitions.

General architectures for LLM agents already cover guardrails, runtime recording, and provenance [8] and registries, adapters, and evaluators [9]. Skill-focused work covers selection, permission-checked binding, and lifecycle governance in reviews and a lifecycle-staged reference architecture [2, 10, 11]. These works organise mechanisms by topic or lifecycle stage. We synthesise documented practices into architectural patterns and a reference architecture (RA) organised by the objects that skill decisions govern. The

boming.xia@adelaide.edu.au (B. Xia); liming.zhu@csiro.au (L. Zhu); zhenchang.xing@csiro.au (Z. Xing); qinghua.lu@csiro.au (Q. Lu); dino.sejdinovic@adelaide.edu.au (D. Sejdinovic); xiwei.xu@csiro.au (X. Xu)

ORCID(s): 0009-0003-7385-4023 (B. Xia); 0000-0001-5839-3765 (L. Zhu); 0000-0001-7663-1421 (Z. Xing); 0000-0002-9466-1672 (Q. Lu); 0000-0001-5547-9213 (D. Sejdinovic); 0000-0002-2273-1862 (X. Xu)

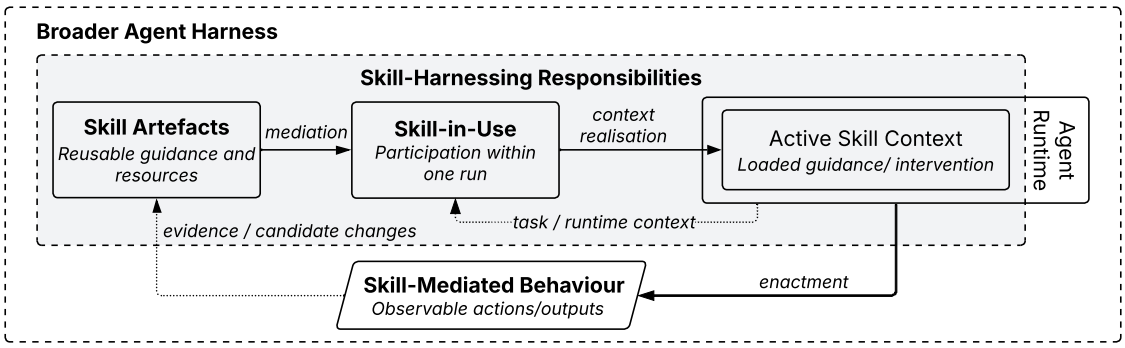


Figure 1: From skill artefact to skill-mediated behaviour. Mediation decides whether an artefact participates in a run as skill-in-use; context realisation supplies its guidance, resources, or intervention as active skill context in the agent runtime; enactment produces the resulting behaviour, whose evidence can yield candidate changes to the artefact.

RA specifies how a skill's declarations enter the host's permission decision, how that decision is enforced on each route that runs the skill's code, and how runs and revisions are traced back to the skill, supporting comparison of implementations and concrete requirement checks.

We conducted a multivocal literature review [12] covering 36 practitioner-documented systems and 51 academic papers, with searches in May 2026. Extracted practices informed ten patterns: five core patterns for the central concerns (participation, authority, verification, provenance, and feedback) and five supporting patterns for narrower decisions. We constructed the pattern-oriented RA by adapting Galster and Avgeriou's method for empirically grounded RAs [13], combining pattern solutions, underlying practices, and architectural reasoning. The RA has four responsibility layers (Supply Chain, Mediation, Execution Control, and Evidence & Feedback) and two cross-cutting substrates (Policy/Configuration and Identifier).

We evaluated the RA through mappings of eight systems and focused technical applications in Deep Agents (LangChain) and Hermes Agent (Nous Research), hereafter Hermes. The mappings compare how documented systems realise each responsibility and under which conditions; the applications examine whether selected responsibilities and dependencies translate into concrete configurations whose consequences can be checked. The paper makes two main contributions:

- **Pattern catalogue.** Ten architectural patterns describing skill-centred problems, solution structures, forces, and trade-offs, with representative source support.
- **Reference architecture and its evaluation.** A logical organisation of skill-harnessing responsibilities, their interactions, and variation points, distinguishing skill-specific decisions from shared harness mechanisms. Mappings of eight systems locate most partial or unestablished realisations in Evidence & Feedback; two applications turn RA dependencies into checks that exposed gaps in documented host controls.

2. Conceptual Foundation

The patterns and RA rest on three objects: a persistent skill artefact, its participation within a run, and the behaviour observed during that participation (Figure 1).

2.1. Skill Artefact, Skill-in-Use, and Skill-Mediated Behaviour

Skill artefact. A skill artefact is an identifiable, persistent unit of behavioural guidance or intervention logic stored outside model weights, together with any associated metadata and resources. The host supplies or applies it as an individual, reusable unit through selection, invocation, an activation condition, or configuration. Its representation can be textual [6], executable [5], or both.

Skill-in-use. Skill-in-use is the participation of a skill artefact in a particular run, such as a task or session whose boundaries the analysis specifies. Participation can begin through invocation or preloading, or carry over from an earlier run. Within the agent runtime, it is represented by the *active skill context*: the loaded instructions, available resources, or intervention state.

Skill-mediated behaviour. Skill-mediated behaviour comprises observable actions and outputs linked to skill-in-use by run evidence, such as tool calls, generated files, or changes to proposed actions.

Anthropic's PDF skill illustrates the three objects [3, 16]. Its package, which includes `SKILL.md`, `forms.md`, and helper scripts, is the skill artefact. In the published walk-through, Claude reads `SKILL.md` through a Bash tool call and then reads `forms.md`. This staged loading is the skill-in-use for the task, and the loaded instructions form its active skill context. The reads themselves are skill-mediated behaviour: they show that the files were loaded, not that Claude carried out their PDF-processing instructions.

2.2. Skill Artefacts and Related Forms

Skill artefacts interact with several related forms, and one implementation can combine their roles (Table 1). Hosts supply skill artefacts in different ways: Claude Code can load named skills into a configured subagent at startup [17],

Table 1
Related forms and their architectural roles.

Form	Contribution and runtime role	Relation/Distinction to skill artefacts
Model weights	Parametric capabilities used during inference	Internal to the model; skill artefacts are represented outside them
Reusable prompts, instruction files	Textual guidance loaded into model context	Can implement a skill artefact when a host supplies it as an identifiable, reusable behavioural unit
Retrieved context and memory	Facts, experience, or behavioural guidance supplied as context	Can supply skill artefacts as well as ordinary contextual information
Callable tools, code, scripts	Callable operations executed by a runtime or service	Can implement a skill's intervention or helpers
Workflows, plans	Task structure and coordinated steps used by an agent or orchestrator	Can organise skill use within a larger execution process
Plugin frameworks	Extension and distribution mechanisms	Can distribute skills alongside other extensions
Model Context Protocol (MCP) [14]	Interfaces to operations and resources	Can expose capabilities used by skills; its Skills extension can also supply skill artefacts [15]
Skill artefacts	Reusable guidance or intervention logic loaded or applied by the host, with associated resources	One host-supplied unit can combine guidance, executable resources, and, in some formats, permission declarations

and OpenHands supplies path-triggered rules and repository instructions such as `AGENTS.md` through its skill mechanisms [18]. Managed instruction files are therefore skill artefacts whether they are loaded on demand or, like `AGENTS.md`, in every run.

2.3. Architectural Concerns in the Artefact-to-Use Transition

Each step from stored artefact to observed change raises its own concerns. Supply raises trust and admission, which the supporting patterns address (Section 4.6). The five concerns below arise once a skill enters a run; each refines a concern shared with broader harnesses.

Runtime participation. A skill's descriptor, such as its name and description, can be exposed before activation loads its instructions or applies its intervention, and supporting resources may be accessed later still [5, 6]. These stages distinguish availability and selection from activation and use.

Skill declarations and execution authority. Execution authority means the permissions applicable to an action, as enforced by host and resource controls. Naming a capability in guidance does not grant permission to use it, but hosts can interpret skill declarations as permission inputs. In Claude Code's documented `commit` example, the `allowed-tools` field lists `git add`, `git commit`, and `git status`. When host policy honours it, these commands are pre-approved only for the invoking turn, while the skill's instructions can remain in context [19]. Instruction and permission lifetimes can therefore differ.

Enactment and verification. Verification criteria must match the enactment mechanism and the claimed outcome: loading guidance does not show that the agent followed it, and executing an intervention does not show that the task succeeded.

Run-specific behavioural provenance. A skill inventory shows which skills were available, whereas provenance records which of them participated in a particular run. It links artefact identity and version with participation, permission decisions, and observed behaviour. Such links establish association, not causal contribution to task outcomes.

Run-to-artefact feedback. Run-local repair changes skill-in-use within the current run, whereas persistent evolution revises the artefact supplied to future runs. Because a persistent revision can also reach other users of the artefact, its review must consider consequences beyond the run that prompted it.

The patterns in Section 4 address these concerns, and the RA in Section 5 organises their responsibilities and their connections to the surrounding harness.

3. Methodology

We developed the pattern catalogue (Table 2) and RA through three stages (Figure 2). First, we planned the review and piloted it: two researchers independently assessed a pilot sample of 12 systems to refine the assessment criteria and extraction schema, resolving disagreements through discussion, and preliminary candidate patterns from the pilot informed later coding. Second, a multivocal literature review (MLR), guided by established methods [12, 20], identified documented skill-harnessing practices and consolidated them into patterns. Third, we constructed an empirically grounded RA from patterns, source evidence, and architectural reasoning (Section 3.2). After the pilot, the first author assessed the remaining sources, constructed the candidate patterns, consolidated the catalogue, and developed the RA. Ambiguous cases were discussed among coauthors and resolved by consensus.

3.1. Multivocal Literature Review

The MLR combined first-party technical documentation of agent systems with academic accounts of skill-related practices. The supplementary materials provide the source,

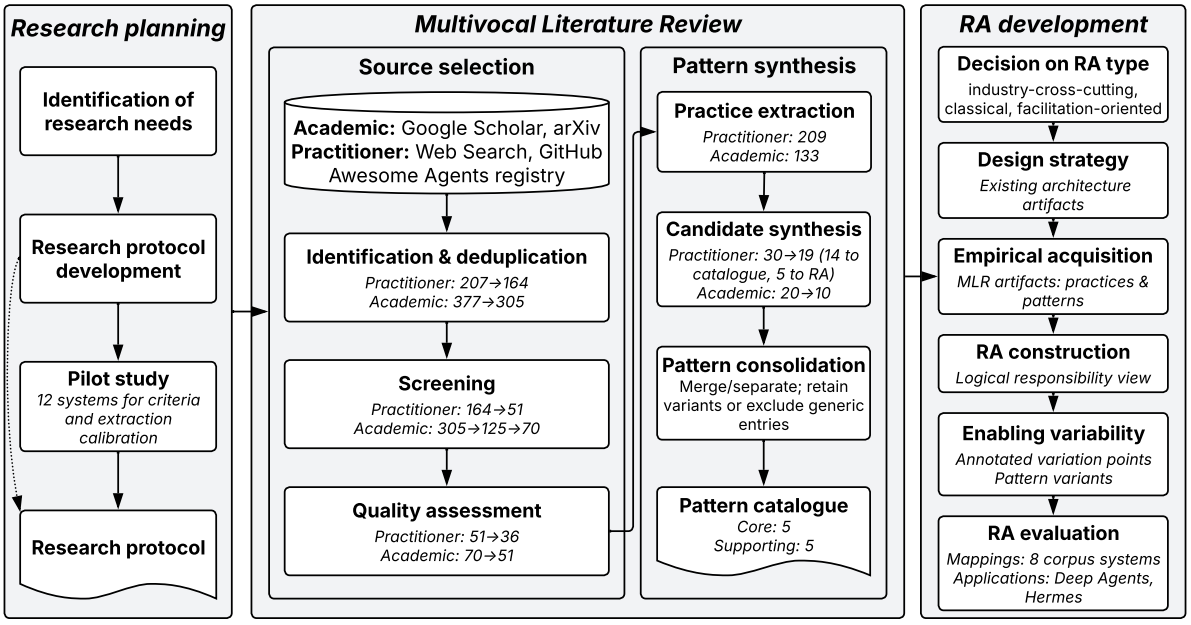


Figure 2: Methodology, with RA development adapted from the six-step method of Galster and Avgeriou [13]. Arrows give counts before and after each stage.

practice, synthesis, and RA derivation records, and the application protocols, drivers, and results[1].

3.1.1. Source Identification

For practitioner sources, we screened 137 entries from the *Awesome Agents* registry and 13 results of the web query `(Agent OR Agentic) AND (Skill OR Skills)`. Follow-up searches targeting named systems and skill-documentation terms identified 54 further records; following first-party links added 3. Searches were conducted on 13 May 2026. Search results identified systems; extraction used first-party documentation and repositories. Deduplication and removal of non-system records reduced the 207 records to 164.

For academic sources, we searched Google Scholar and arXiv on 8 May 2026 for records from 2025 onward. The Google Scholar query was `allintitle:(LLM OR LLMs OR "Large Language Model" OR "Large Language Models" OR Agent OR Agents OR Agentic OR Harness) AND (Skill OR Skills)`, with an equivalent title-field query in arXiv. This precision-oriented strategy targeted contemporary agent skills: even the title-restricted searches returned uses of "skill" for human training, robotics, and model competence. Google Scholar returned 165 records and arXiv 207; targeted snowballing from four seed papers added 5. Deduplication retained 305 unique academic records.

3.1.2. Eligibility and Detailed Assessment

Both practitioner and academic streams applied the same eligibility criteria. A source was eligible if it described a reusable, externally represented, agent-facing skill artefact, as characterised in Section 2, together with a mechanism that managed, governed, recorded, or revised its participation in a run. Sources describing only generic orchestration, tools, plugins, package management, or prompt templates without such a managed artefact were excluded, as were skill literatures concerned with agent-internal learned policies or action repertoires. Eligible sources needed sufficient technical detail to identify a concrete mechanism and its architectural responsibility. Practitioner evidence had to be first-party.

For practitioner sources, initial keyword screening was followed by skill-signal triage: checking first-party README or documentation pages for skill-related terminology tied to a reusable agent-facing artefact. This retained 51 practitioner sources for detailed assessment: 22 from the registry and 29 from web searches. Detailed assessment considered fit with agent-system skill harnessing, duplicate product lineages, maintenance, adoption, and architectural detail beyond already retained sources. Repository activity within the preceding six months informed maintenance assessment; current first-party technical documentation could also establish currency. We retained 36 practitioner systems.

For academic sources, title and abstract screening retained 125 of the 305 records for eligibility checking: 70 advanced to full-text assessment, 44 became background records without practice extraction, and 11 were excluded. Full-text assessment rated technical specificity, practice extractability, and evidence quality (implementation and evaluation detail), and considered study type and transferability beyond one application domain; it retained 51 papers for extraction, classified 17 as background, and excluded 2.

3.1.3. Practice Extraction

A practice is a source-supported mechanism or architectural decision concerning a skill artefact or its use. Each practice record linked a located source passage to a practice

[1] https://github.com/bomingxia/harnessing-agent-skills-supplement

description, a descriptive mechanism category, and boundary or audit notes. The practitioner and academic streams yielded 209 and 133 practice records.

3.1.4. Pattern Synthesis and Consolidation

Within each stream, we grouped practices by architectural problem and responsibility-level solution, producing 30 practitioner-stream and 20 academic-stream candidate patterns from the 342 practices. Whether to retain, merge, or exclude each candidate was a qualitative judgement on recurrence, reusability, forces and consequences, traceable support, and what the skill artefact changed about the decision, with the reasons recorded.

The practitioner stream kept 19 of its 30 candidates: 14 for the catalogue and five that inform RA responsibilities, supplying the basic artefact unit and enabling functions such as supply and enforcement (Table 3). The other 11 were not retained: five adjacent or general mechanisms, of which generic permission and sandbox policies later inform Runtime Action Control; five non-pattern proposals, such as command syntax; and one inventory-and-preview component. The academic stream consolidated its 20 candidates into ten intermediate entries. The 14 practitioner candidates and the ten academic entries were then consolidated into the catalogue. Within and across streams, we merged equivalent problem–solution structures, such as the activation candidates; separated distinct responsibilities, such as run-local repair and persistent evolution; and absorbed narrower features as variants, such as parameterised invocation.

The catalogue contains ten patterns: five core and five supporting. Representative support and qualifications are reported with each pattern in Section 4, and Section 5.1 traces one candidate from source practices to the RA.

3.2. Reference Architecture Construction

We adapted the six-step method of Galster and Avgeriou [13] by placing the pattern catalogue between the documentary evidence and the logical architecture description. The method distinguishes the empirical foundation established through Steps 1–5 from the evaluation in Step 6.

Step 1 (Decision on RA type). In usage context, the RA is industry-cross-cutting: it organises common skill-harnessing responsibilities for agent-system designers and implementers across application sectors, organisations, and platforms. It is classical because it draws on experience from existing systems, and facilitation-oriented because it provides guidance for architectural design and analysis [21].

Step 2 (Selection of design strategy). The design strategy is generalisation from existing architectural evidence. Patterns supply reusable problem–solution structures; the underlying practices ground these abstractions and inform the enabling functions needed for their interaction. This pattern-oriented strategy is consistent with earlier reference architecture work [8].

Step 3 (Empirical data acquisition). RA construction used the extracted practices, their pattern solutions, and documented shared harness mechanisms (Sections 3.1.4 and 4.6), with the concerns in Section 2.3 as architectural requirements.

Step 4 (RA construction). We derived responsibilities from pattern solutions and practice records and grouped them into blocks around a shared object or decision boundary: scope resolution and eligibility share a block because both determine the selectable pool, whereas permission determination and enforcement, like evidence capture and verification, form separate blocks. Functions that pattern solutions presuppose, such as artefact supply, form enabling blocks; rules and identifiers used across blocks form substrates. Layers group blocks by the object they govern: supplied artefacts, runtime participation, action authority, and run evidence and change. Interactions link blocks through documented or inferred input–output dependencies. Block names state the grouped decisions, as in Resolution & Admission, or the enabling function, as in Distribution. Section 5 specifies these conventions as a *Responsibility Viewpoint* following ISO/IEC/IEEE 42010 [22] and presents the resulting Responsibility View. Derivation records, compiled after construction, link each allocation and dependency to its source evidence and reasoning.

Step 5 (Enabling variability). We derived variation points from documented pattern variants and differences among systems. These are represented as textual annotations describing alternative realisations of responsibilities, while preserving their dependencies (Section 5.5).

Step 6 (RA evaluation). We assessed the RA through documentary mappings of eight review-corpus systems and focused technical applications, reported in Section 6.

4. Architectural Patterns for Skill Harnessing

Table 2 summarises the ten patterns. The five core patterns address the five concerns of Section 2.3 (Sections 4.1 to 4.5); the five supporting patterns address narrower decisions, including trust and admission, that realise or complement them. Each core pattern states its intent, problem, forces, solution, variants, consequences, and evidence. The pattern descriptions also draw on current host documentation and sources outside the review corpus to illustrate mechanisms and variants.

4.1. Progressive Skill Activation

Intent. Stage the transition from available skill artefacts to active skill-in-use, exposing or applying what is required at each stage (Figure 3).

Context and problem. Agent systems may expose many skills from various sources. Loading all skill bodies into a run can consume context unnecessarily and introduce irrelevant guidance. Hiding all skill information until invocation creates the opposite problem: the agent or user cannot identify relevant reusable behaviour. The architectural problem is to provide enough information for selection while controlling when full guidance, resources, or interventions participate.

Table 2
Catalogue of skill-harnessing architectural patterns.

Pattern	Problem and solution in brief
	Core patterns
Progressive Skill Activation	Available skill artefacts may influence behaviour or consume context if their guidance is exposed too early in a run. Hosts expose compact descriptors first and load full guidance and resources only on activation.
Skill–Execution Authority Separation	Skill declarations can affect permissions when interpreted by a host. Host and resource policy decides each action's permissions; skill declarations are inputs to that decision, separate from skill participation.
Verifiable Skill Contract†	Loading or invoking a skill does not show that it was followed or succeeded. A verifier checks skill-scoped criteria against run evidence and records a verification status.
Runtime Skill Bill of Materials†	Repository inventories do not identify which skill artefacts and versions participated in a run. A skill-centred record links participation decisions to run evidence.
Skill–Agent Co-Evolution Loop	Run evidence may suggest skill changes. Candidate changes require validation before entering the shared skill supply.
	Supporting patterns
Skill Eligibility Gate	Supplied skills require assessment for admission to the selectable pool. A gate applies validation, trust, or compatibility conditions.
Skill Scope Cascade	Skills defined at several scopes, such as organisation, user, and project, can expose unintended guidance or resources. Visibility and precedence rules determine which skills are visible and resolve competing artefacts.
Skill Resource Materialisation Boundary	Reading a skill's files and running its helpers have different costs and permissions. An explicit materialisation decision identifies which resources are supplied and which helper operations occur.
Skill Extension Bundle	Guidance and its supporting material can be distributed separately. A deployable bundle gives them coordinated distribution, compatibility, and review boundaries.
Runtime Skill Repair†	A skill may fail verification or skill-scoped enactment criteria mid-run. Bounded adaptation of skill-in-use lets the run continue without changing the shared artefact.

†Known uses are mainly research systems; practitioner systems document partial mechanisms.

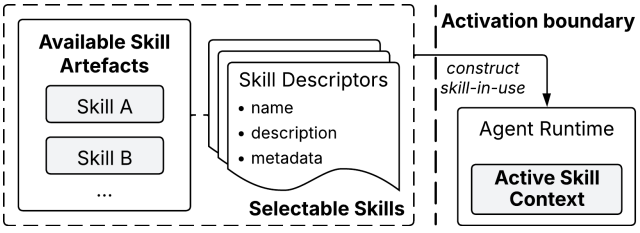


Figure 3: Progressive Skill Activation: in the descriptor-first realisation shown, compact descriptors support selection and activation supplies instructions as active skill context.

Forces. *(i) Discoverability vs. premature full exposure.* A skill must be visible enough to support selection, but exposing full guidance too early consumes context [23] and may influence the run before the skill is needed. *(ii) Coverage vs. activation precision.* Large skill inventories improve reuse coverage but also increase selection ambiguity and the risk of activating irrelevant or conflicting guidance. *(iii) Traceability vs. operational simplicity.* Inspecting skill influence supports review, repair, and revocation, but finer-grained records increase maintenance cost.

Solution. Activate skills through staged architectural states. A skill artefact is *available* when present in a source the system can inspect. It is *selectable* when the host exposes it for selection, commonly through a compact descriptor. It is *active* when invocation, configuration, or an activation condition supplies its guidance or applies its intervention in the run. Where supporting material is required, activation remains separate from resource materialisation. Descriptor-first disclosure keeps candidate skills discoverable while delaying full instruction loading. Configured preloading can activate a named skill at startup, and programmatic skills can activate through a predicate.

Variants. Variants differ in who initiates activation. In *explicit activation*, a user, command, prompt mention, or interface action requests that a selectable skill become active. In *policy-mediated activation*, the system activates selectable skills according to configuration, scope, triggers, or applicability predicates. In *agent-proposed activation*, the agent proposes a skill from exposed descriptors, while the system decides whether activation is permitted. Activation may occur at run start or be re-applied as a task proceeds.

Consequences. *(i) Controlled exposure.* Full guidance or interventions enter through defined activation mechanisms; descriptors can already influence selection. *(ii) Lower context cost.* Descriptor-first exposure delays full instruction loading until needed. *(iii) Improved attribution.* Available, selectable, and active skills can be logged separately. *(iv)*

Added coordination burden. Descriptors, activation policies, and state transitions must be maintained consistently.

Evidence. Progressive disclosure from descriptor exposure to body loading is realised in Claude Code, OpenHands, Hermes, and OpenAI Codex. In the PDF example in Section 2.1, metadata makes the skill discoverable before Claude reads `SKILL.md` and then `forms.md`. This sequence exposes instructions and references progressively, preserving discoverability while deferring their context cost.

Hosts also differ in how activation begins. Explicit activation includes Codex's `/skills` menu or `$skill-name` invocation. Policy-mediated activation appears in OpenHands' keyword-triggered skills, which load when a prompt contains a configured trigger. In Claude Code, descriptions guide agent-proposed selection; configured skill preloading can instead activate named skills at subagent startup [17].

4.2. Skill–Execution Authority Separation

Intent. Separate skill guidance and capability declarations from the host's determination of action permissions (Figure 4).

Context and problem. Skills often refer to tools, APIs, MCP servers, scripts, services, or files as task resources, and hosts can interpret skill declarations as permission inputs. Execution authority may also exist independently of any active skill. The architectural problem is to relate a skill's capability requirements and declarations to the permissions applicable to each action.

Forces. *(i) Capability expression vs. authority granting.* A skill artefact may need to express expected capabilities, but treating prose references as permission grants confuses stated needs with authorised use. *(ii) Portable guidance vs. contextual authority.* Skill artefacts should be reusable across agents and workspaces, but tools, credentials, approval rules, and side-effect policies are context-specific. *(iii) Independent change vs. runtime alignment.* Skill artefacts and authorisation rules may change through different channels, creating runtime mismatches between what a skill expects and what the agent is authorised to do.

Solution. Separate skill-exposed behavioural guidance from the policy decision governing capability use. A skill artefact may describe requirements or carry pre-approval declarations, which the host interprets alongside agent, workspace, platform, user, and resource rules. At action time, the host determines the applicable permissions, including the scope and lifetime of any skill-derived grant. It permits, blocks, or escalates the requested capability use accordingly.

Variants. Variants differ in where the skill–authority separation is resolved. In *skill-scoped pre-approval*, host-recognised declarations attached to a skill contribute permission grants with a host-defined scope and lifetime. In *agent- or profile-scoped binding*, reusable skills operate under an agent, user, workspace, or profile policy. In *platform-mediated binding*, capability access is governed through platform or administrator controls. A research-informed variant is *verification-conditioned binding*, which applies

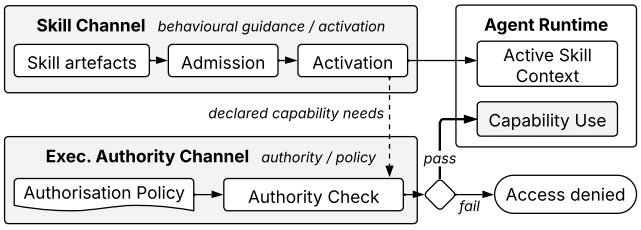


Figure 4: Skill–Execution Authority Separation: host policy determines action permissions and can interpret skill declarations as permission inputs. The check applies to each requested capability use; declared needs are one input alongside policy.

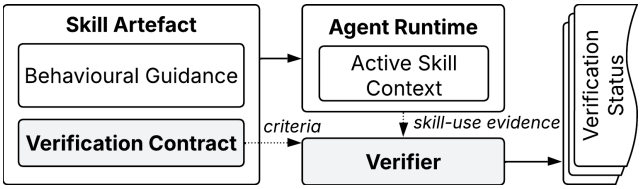


Figure 5: Verifiable Skill Contract: skill-scoped criteria are checked against skill-use evidence by an independent verifier.

stricter treatment according to skill verification state or side-effect class. Skill-carried restrictions form another variant: Claude Code's `disallowed-tools` removes tools for the invoking turn, while invoked skill hooks can persist through the session [19].

Consequences. *(i) Portable guidance.* Skills can describe capability needs while effective permissions are governed in the deployment context. *(ii) Independent authority governance.* Tool and action permissions can be granted, narrowed, revoked, and audited independently. *(iii) Explicit mismatch handling.* Unavailable or unauthorised capabilities are exposed for runtime handling. *(iv) Dual-surface maintenance burden.* The skill artefact's capability metadata and the authority policy require coordinated maintenance.

Evidence. Skill-scoped pre-approval appears in Claude Code and GitHub Copilot CLI through `allowed-tools`. Copilot CLI's documented image-conversion skill illustrates the scope of such a declaration: it packages `convert-svg-to-png.sh` and can pre-approve `shell`. This reduces confirmation prompts, but broad shell approval extends the trust decision beyond one conversion script. Accepting the declaration therefore calls for reviewing both the guidance and referenced executable material [7].

Claude Code's subagents combine preloaded skills with separately configured tool permissions [17]. By contrast, OpenCode's skill-access rules and Codex's invocation policy govern whether a skill can be loaded or invoked. Research on verifiable artefacts extends authority decisions to verification state and side-effect class, supporting the verification-conditioned variant [24].

4.3. Verifiable Skill Contract

Intent. Associate a skill artefact with independently checkable criteria for assessing its use and producing an explicit verification status (Figure 5).

Context and problem. An agent may follow, partially follow, reinterpret, or overextend skill guidance; an executable intervention may run without achieving its intended effect. The architectural problem is to check declared skill-scoped criteria against evidence of the relevant actions, states, or outputs.

Forces. *(i) Context flexibility vs. verifiability.* Skills must remain useful across tasks and contexts, but verification requires criteria precise enough to assess enactment. *(ii) Independent checking vs. verifier burden.* Checking outside the agent's output path reduces self-confirming success claims, but makes verifiers, tests, replay environments, or assertion evaluators part of the trusted architecture. *(iii) Assurance strength vs. contract brittleness.* Weak criteria provide limited assurance; overly rigid contracts may reject valid context-sensitive behaviour or become costly to maintain.

Solution. Associate a verifiable contract with the skill artefact and introduce a verifier as a separate checking role. The contract specifies the criteria to assess, such as interface constraints, expected state changes, assertions, test cases, or replay checks. The verifier evaluates the corresponding evidence outside the skill's self-report path and produces a status such as *passed*, *failed*, *inconclusive*, or *verifier error*. The status identifies the assessed criteria and is available to downstream decisions.

Variants. Variants differ in the checking point and evidence used. In *boundary checking*, verification occurs at an invocation or interface boundary through typed arguments, schemas, preconditions, postconditions, or output constraints. In *trajectory checking*, assertions are attached to intermediate steps or states. In *execution- or replay-based checking*, the skill artefact or candidate skill is run against reference tasks, replay bundles, or regression tests. These offline checks assess the candidate under the tested conditions; checking a current run requires that run's own observations.

Consequences. *(i) First-class skill-use status.* Verification produces a structured status alongside the output or trace. *(ii) Conditioned composition.* Downstream processes can condition continuation on verification status. *(iii) Targeted repair signals.* Failed clauses or replay diagnostics can guide repair or evolution. *(iv) Verifier governance burden.* Contracts and verifiers become artefacts that must themselves be maintained, versioned, and trusted.

Evidence. Typed arguments for atomic computer-use skills [25] and typed intermediate representations [26] specify structured invocation and representation interfaces. Checking intermediate execution states requires corresponding observations. ContractSkill provides this connection: each web-skill step can carry post-assertions evaluated against page evidence, alongside declared goals, preconditions, postconditions, recovery, and termination checks [27]. A failed assertion identifies the step-level criterion that was not met, giving a more specific repair signal than final task failure alone.

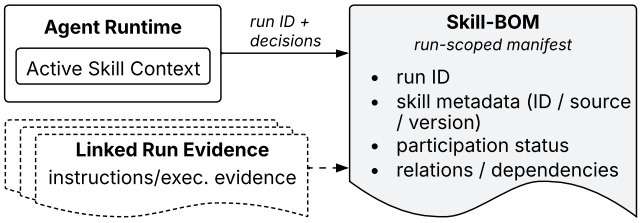


Figure 6: Runtime Skill-BOM: the proposed per-run, skill-centred provenance record links skill artefacts, participation decisions, and run evidence.

Offline checks before candidate acceptance appear in execution-verified programmatic skill induction [28] and FlowEvo's interface, safety, and replay validation, with replay checks applied where execution is feasible [29]. In the HASP skill-program framework, syntax, interface, mock-execution, and return-type checks similarly assess programmatic candidates before they enter the library [5].

4.4. Runtime Skill Bill of Materials (Skill-BOM)

Intent. For a specific agent run, record the relevant skill artefacts, their source and version metadata, participation decisions, and operational skill-to-skill relations (Figure 6).

Context and problem. Software Bills of Materials make component provenance and dependency information explicit for supply-chain assurance [30]. A skill repository exposes available artefacts, while a run involves particular versions and participation decisions: what was exposed, selected, activated, excluded, unavailable, or used in a composition. Relevant records may be scattered across inventories, configuration, loader events, and execution traces. The architectural problem is to join these records into a run-specific view of identifiable skill artefacts and their participation.

Forces. *(i) Run specificity vs. inventory overbreadth.* A repository inventory describes skill supply. Attribution requires identifying the artefacts and decisions associated with the analysed run. *(ii) Attribution fidelity vs. evidence overhead.* High-fidelity attribution depends on linking evidence across supply, activation, policy, and runtime, but such linkage increases collection and correlation cost. *(iii) Skill-centred provenance vs. audit-log sprawl.* A complete run log may contain relevant events, but skill attribution remains brittle if skill identities, versions, activation decisions, and relations are scattered across generic traces.

Solution. Maintain a run-scoped *Skill-BOM* as an explicit manifest or reconstructed skill-centred record over run evidence. Entries link a run identifier to skill artefact identities, source and version information, and participation events such as exposure, selection, activation, or exclusion. Relevant permission decisions link to the actions and times they govern. Where skill relations affect the run, record the relevant dependency, composition, or load-triggering edges. SkillNet provides related dependency and composition vocabulary [31].

Variants. Two implementation options differ in how the record is produced. In an *explicit run manifest*, the record is

stored as a bounded artefact or snapshot. In a *reconstructed manifest*, the record is assembled from linked loader, activation, configuration, runtime, test, review, or update records through shared run and skill identifiers.

Consequences. *(i) Skill-level attribution.* Developers can distinguish installed skills from skills exposed, activated, excluded, or otherwise relevant to a run. *(ii) Targeted review and maintenance.* Findings can be linked to specific skill artefacts, versions, sources, relations, or activation decisions. *(iii) Provenance burden.* The pattern requires stable run, skill, version, relation, and decision identifiers. Incomplete records can create false assurance.

Evidence. The complete Skill-BOM is a proposed architectural synthesis of documented invocation, identity, integrity, and provenance mechanisms. Artefact identity is supported by skill manifests with stable identifiers for registry indexing [32], lockfiles and an installation-level agent skill bill of materials [33], and provenance records carrying documentation hashes and skill identifiers [34]. Invocation-time checking connects such records to the loaded content: Shen et al. [35] verify loaded content against audited registry records and, at load time, limit the injected execution envelope to approved manifest permissions within the user's authorised scope.

Invocation records provide the corresponding runtime information. Snowflake Cortex Agents monitors the selected skill, input, and result, while its configuration identifies Snowflake stage or Git sources. Joining an invocation to the resolved skill version brings these records into a run-specific provenance view. Skill-linked trajectory annotations add file-level detail: Liu et al. [36] record subtasks with file paths and line ranges. The record can also incorporate Aeon's per-skill quality scores, failure flags, and health history, and replay or contract evidence bundles [37].

4.5. Skill–Agent Co-Evolution Loop

Intent. Couple agent execution with governed revision of reusable skill artefacts: accepted skills shape future runs, and run evidence informs changes.

Context and problem. Skill artefacts shape runs through activation, while run evidence can inform artefact revision. Missing steps, repeated workarounds, tool failures, user corrections, and successful procedures provide candidates for improvement; Zhao et al. [38] distil successful strategies and failure-mode fixes into reusable skills. A feedback path captures these candidates for refinement and validation: local fixes can overfit a single task or propagate regressions across agents. The architectural problem is to turn run experience into reusable improvements while controlling changes to shared artefacts.

Forces. *(i) Cumulative learning vs. behavioural stability.* Skills should improve from accumulated run experience, but shared behavioural artefacts must remain stable enough to be trusted across contexts. *(ii) Local evidence vs. generalisable change.* Run evidence may reveal a useful procedure, but skill-mediated behaviour depends on specific context and evidence from one context may not generalise. *(iii) Automation scale vs. acceptance cost.* Frequent skill updates increase manual maintenance effort, while automated proposals still require validation before changing shared guidance.

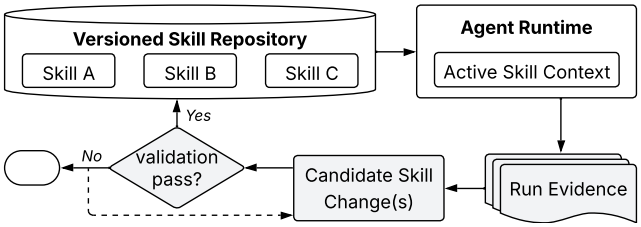


Figure 7: Skill–Agent Co-Evolution Loop: accepted skills shape runs; run evidence produces candidate changes whose acceptance is governed by validation; rejected candidates do not enter the repository.

Solution. Introduce a validated feedback loop between agent runs and the skill repository (Figure 7). The forward path activates accepted skill artefacts in later runs. The reverse path turns run evidence into candidate skill creation or refinement. Validate candidates before updating shared artefacts, then version accepted changes and return them through managed supply for future activation.

Variants. Architectural variants differ mainly in where run-derived skill changes are made reusable. In *session- or workspace-scoped evolution*, a session, project, or workspace turns its own traces, conversations, failures, or successful procedures into persistent skill changes for later local reuse. In *repository-governed evolution*, candidate changes are routed through a managed repository for evaluation, deduplication, promotion, or retirement. In *collective evolution*, evidence from multiple agents, users, or sessions is aggregated before validated updates are synchronised into a shared skill set.

Consequences. *(i) Cumulative improvement.* Repeated procedures and corrections can become reusable skill artefacts. *(ii) Reviewable change.* Candidate changes can be inspected, accepted, or rejected; accepted versions can be rolled back. *(iii) Validation bottleneck.* Weak validation promotes drift while strict validation slows useful updates. *(iv) Evolution provenance burden.* The loop requires records linking evidence, candidates, validation verdicts, accepted versions, and later runs.

Evidence. Feedback-driven skill revision is recurrent, while validation, promotion, and shared governance vary across realisations. Skill1's trajectory distillation and utility updates [39] illustrate how run experience feeds reusable skills, with related autonomous skill revision from execution experience in XSkill [40] and SkillWeaver [41].

How those changes are accepted varies. FlowEvo compiles successful workflows into validated executable skills [29], whereas Trace2Skill's default path consolidates trajectory-derived patches into a skill directory without validation-based patch selection [42]. Repository governance makes the responsibility for change explicit: SkillOS separates a frozen executor from a curator that edits

the skill repository [43], while Aeon routes verification-triggered changes through reviewable pull requests with attached verification plans. Skill-bank operations and retention replay provide further mechanisms for managing reusable changes [37, 44]. The feedback loop can also aggregate experience beyond a single run or user, as in SkillClaw's multi-session evidence and shared skill updates [45].

4.6. Supporting Patterns

Each entry states the problem, architectural decision, central force, and supporting evidence.

Skill Eligibility Gate. This pattern places an admission boundary before selectability, upstream of Progressive Skill Activation: supplied artefacts enter the selectable pool when declared conditions such as validation, trust, enablement, or applicability hold. Because descriptors can steer selection before instructions load, admission governs their exposure. The central force is open reuse versus controlled exposure: a broader pool increases discoverability, while admission criteria restrict which units can participate. These admission conditions include enable/disable toggles in Cline and OpenAI Codex, project trust in Hermes, and load-time filters in OpenClaw. Where admission depends on validation, checks such as PraisonAI's structural and health checks provide inputs to the decision.

Skill Scope Cascade. Skill artefacts may exist at multiple scopes, from system or organisation level to project, workspace, user, file-path, or agent level. This pattern resolves which scoped artefacts are visible and, where the host applies precedence, which instance is selected. It specialises configuration resolution for reusable guidance or intervention logic: selecting an unintended artefact can affect subsequent execution. The central force is contextual locality versus governing authority: local skills may better fit the task, while higher-level scopes may encode shared policy, compatibility, or safety constraints. Source precedence resolves competing artefacts. It appears in enterprise/personal/project precedence in Claude Code, which also supports namespaced plugin skills and nested discovery; multi-location precedence in OpenClaw; global/project precedence in Cline; and workspace/global precedence in Kiro. Visibility determines which artefacts a recipient can access, as illustrated by Cursor Agent's project/user directories and nested or path-scoped skills and CrewAI's crew-level skill binding. A host can combine precedence and visibility when resolving which skills are visible.

Skill Resource Materialisation Boundary. Skill artefacts may depend on supporting material such as reference files, templates, examples, assets, scripts, or runtime context. Deciding that a skill participates leaves a further decision about which resources to supply and how to obtain them. This pattern makes resource materialisation explicit: reading a reference, resolving an asset, and generating context with a helper have different costs and executable effects. The central force is contextual fidelity versus controlled expansion: task-specific resources can improve the supplied context, while access and execution require appropriate permissions and provenance. Reference access appears in Claude Code's supporting files and Cline's referenced resources, while Copilot CLI bundles executable scripts. Materialisation can also occur within invocation: Claude Code's dynamic context injection runs commands before sending the rendered skill content to the model. Here, supplying context involves executable operations, connecting *Progressive Skill Activation* to the capability controls of *Skill–Execution Authority Separation*.

Skill Extension Bundle. Skills can depend on executables, integration code, hooks, MCP-server configuration, or other settings governed through separate review and authority channels. This pattern brings guidance and supporting material into a deployable bundle, so review covers both instructions that direct execution and the code they invoke. It applies across runs, complementing the per-run *Skill Resource Materialisation Boundary*. Separately supplied dependencies require compatible versions and explicit references. The central force is deployment cohesion versus review-surface expansion: bundles simplify sharing and coordinated updates, while admission and trust review cover a broader unit. Co-delivery is realised through plugins that package skills with hooks, MCP server configuration, and other components in Claude Code and OpenAI Codex, and through plugin or extension skills in OpenClaw, Qwen Code, and Goose.

Runtime Skill Repair. A skill-in-use may fail verification or skill-scoped enactment criteria during a run. The architectural question is whether execution can continue through a bounded change to its current bindings, composition, or runtime representation. This pattern permits same-run repair constrained by declared checks, locality, and a repair budget. The adapted object is skill-in-use, while the stored artefact and its version remain unchanged. The central force is run continuity versus behavioural preservation: repair changes the current execution arrangement while preserving unaffected verified work. It can be triggered by *Verifiable Skill Contract*. GraSP provides direct evidence for bounded execution-graph repair: patches preserve graph validity and unaffected verified nodes, and newly introduced nodes use library skills [46]. SkillTracer provides related evidence for selective repair of attributed plan graphs and also supports persistent refinement [47]. Its documented address-editing case expects a `combobox` for state selection but encounters a `textbox`; the failed node is localised, the valid prefix preserved, and execution resumed after repair and re-verification.

Beyond these patterns, the evidence base contains broader harness mechanisms that support skill-mediated runs, such as action approval, sandboxing, workflow composition, plugin and package installation, and lifecycle hooks. The patterns specialise them for skill artefacts, and the RA includes those the patterns presuppose as enabling blocks (Table 3).

5. Reference Architecture for Skill Harnessing

The RA organises the patterns in Section 4 and their underlying practices into a system-level Responsibility View (Figure 8).

5.1. Overview and Architectural Rationale

The system of interest is the skill-harnessing part of an agent system. The agent runtime and execution environment are interfacing systems: the runtime uses skill instructions, resources, or programmatic interventions when producing actions, and the environment performs permitted actions and returns observations.

We define a *Responsibility Viewpoint* for agent-system designers and implementers, using the view and viewpoint concepts of ISO/IEC/IEEE 42010 [22]. It asks which functions address the concerns in Section 2.3, how their decisions and outputs depend on one another, and where implementations may vary. Its elements are responsibility blocks grouped into layers, interfacing systems, and cross-cutting responsibilities called substrates. A block represents one responsibility: a related set of decisions or an enabling function; pattern insets mark each pattern's principal allocation. Labelled links express operational interactions or evidence exchange. Concrete components can combine responsibilities or delegate them to shared harness services.

The corresponding *Responsibility View* has four layers, each governing one object: Supply Chain (persistent artefacts, their distribution, and acceptance of revisions), Mediation (runtime participation), Execution Control (skill declarations and execution authority), and Evidence & Feedback (verification, run-specific provenance, and feedback). Their boundaries separate artefact identity from participation, participation from action permissions, and evidence capture from judgements based on it. Policy and identifiers span all layers and form the two substrates.

The activation responsibility illustrates the derivation. Practices from Hermes and OpenAI Codex describe lightweight skill descriptions preceding full instruction loading. They contribute to the practitioner candidate *Progressive Skill Disclosure Loader*, supported by 21 practices from 16 systems. Consolidation with the selection-routing and invocation-policy candidates and academic evidence yielded *Progressive Skill Activation*. With *Skill Resource Materialisation Boundary*, it is allocated to *Activation & Context Realisation* in Mediation: both decide what the runtime receives, whereas Representation & Storage retains artefacts and the Execution Control layer permits actions. Because Claude Code's dynamic context injection runs commands while preparing skill content [19], the RA routes these commands through Runtime Action Control.

The ten patterns have their principal allocation in seven blocks; two further blocks provide enabling functions that these patterns presuppose. Installation, update, sharing, porting, and source-tracking practices motivate Distribution, together with candidate validation and promotion mechanisms [37]. Shared permission and sandbox mechanisms and skill-registered hooks motivate Runtime Action Control, included because skill-initiated operations pass through it, whether a dedicated or a general host gate realises it. Policy coordination and evidence linking motivate the two substrates (Table 3).

5.2. Architectural View

Figure 8 presents the Responsibility View. Converging evidence links collect contributions from the supply, participation, and execution responsibilities and from the interfacing systems; the evidence status of each pattern is given in Table 2.

Interactions impose local ordering constraints: an artefact must be accessible before its guidance can participate, and an executable operation must satisfy the applicable controls before it proceeds. Evidence links the decisions and observations produced across these interactions. Components may perform decisions eagerly, lazily, or together, provided the relevant dependencies are preserved. Table 3 summarises the allocation of patterns and enabling functions.

5.3. Layers and Responsibilities

Layer 1: Supply Chain. *Skill Sources* include AI-generated, user-authored, and third-party artefacts. *Skill Extension Bundle* supplies a packaging option in which guidance and supporting material share a distribution and review boundary. *Distribution* installs and updates artefacts and coordinates the validation and acceptance of persistent revisions, whereas Mediation decides which supplied artefacts become selectable. Where a run must be reproducible or auditable, the identity retained by *Representation & Storage* must resolve to the artefact revision used.

Layer 2: Mediation. *Resolution & Admission* resolves scoped artefacts and determines the selectable pool; skill descriptors can be exposed before activation. *Activation & Context Realisation* establishes participation through invocation, preloading, or activation conditions, combining *Progressive Skill Activation* with *Skill Resource Materialisation Boundary*. The runtime receives active skill context comprising instructions, resources, or intervention state; programmatic interventions can modify proposed actions. Mediation sends recognised skill declarations to Capability Authorisation, materialisation commands to Runtime Action Control before the content reaches the model, and participation events to Run Evidence Capture.

Layer 3: Execution Control. *Capability Authorisation* determines action permissions from host and resource policy, including recognised skill declarations. *Runtime Action Control* enforces the decision alongside approval requirements, sandbox constraints, and side-effect controls. It gates operations proposed by the runtime or initiated during resource materialisation. Permitted operations reach the execution environment, whose observations return to the runtime or materialisation process. The two responsibilities can share a host implementation; *Skill–Execution Authority Separation* governs how skill declarations enter the determination.

Layer 4: Evidence & Feedback. *Run Evidence Capture* links artefact identities and available version information

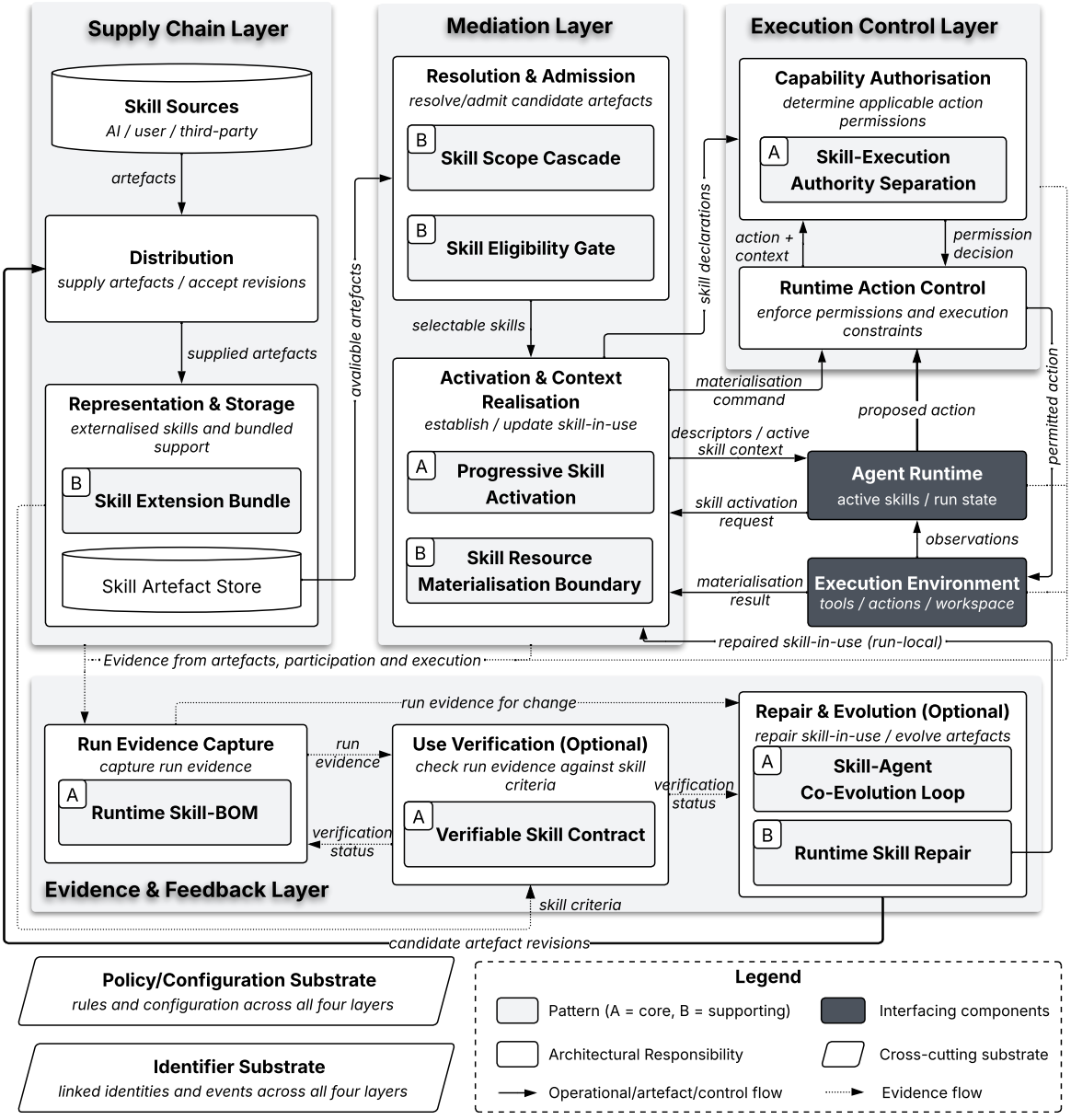


Figure 8: Responsibility View of the reference architecture. Four layers group responsibility blocks by the object they govern; the Policy/Configuration and Identifier substrates apply across all layers. Pattern insets mark each pattern's principal block (A: core; B: supporting). Solid arrows carry operational inputs and outputs; dotted arrows carry evidence, criteria, and results.

with participation, permission decisions, actions, and observations. The proposed *Runtime Skill-BOM* integrates these links into an explicit or reconstructed skill-centred record.

Where verification is required, *Use Verification* checks run evidence against skill-scoped criteria through *Verifiable Skill Contract*. Criteria come from Representation & Storage or the check's configuration. Results return to Run Evidence Capture and can inform Repair & Evolution; results of checks on candidate revisions can accompany them to Distribution for acceptance.

Repair & Evolution has two change objects and return paths. *Runtime Skill Repair* routes bounded adaptations of current skill-in-use to Mediation, with re-verification and without changing the stored artefact. *Skill–Agent Co-Evolution Loop* sends candidate revisions to Distribution, whose acceptance policy specifies validation before they enter the shared supply.

5.4. Cross-Cutting Substrates

The *Policy/Configuration Substrate* supplies the rules consulted for admission, acceptance, activation, authorisation, execution, verification, repair, and evidence retention. These rules can reside in different components while referring to consistent identities and scopes. Reconstructing a past decision additionally requires retaining the applicable policy state or an identifiable policy version with its evidence.

Table 3
RA responsibility blocks and substrates, derivation basis, and architectural purposes.

Responsibility	Derivation basis	Architectural purpose
Supply Chain: Distribution	Enabling function. Practices for installing, updating, sharing, porting, and source-tracking skills; acceptance step of Skill–Agent Co-Evolution Loop	Supplies and updates skill artefacts and governs acceptance of persistent revisions through the required checks or review; retains available source and version information.
Supply Chain: Representation & Storage	Pattern: Skill Extension Bundle. Practices representing skills as stored packages with metadata and resources	Retains reusable artefacts and associated resources; bundling is one packaging option.
Mediation: Resolution & Admission	Patterns: Skill Scope Cascade; Skill Eligibility Gate	Resolves scoped candidates and determines which are eligible to participate.
Mediation: Activation & Context Realisation	Patterns: Progressive Skill Activation; Skill Resource Materialisation Boundary	Establishes participation through instructions, resources, or programmatic interventions.
Execution Control: Capability Authorisation	Pattern: Skill–Execution Authority Separation	Determines applicable action permissions from host and resource policy, including recognised skill declarations and the scope and lifetime of any skill-derived grants.
Execution Control: Runtime Action Control	Enabling function. Practices for permission and sandbox policies and for hooks registered from skill metadata	Enforces permissions and execution constraints on operations proposed by the runtime or initiated during materialisation.
Evidence & Feedback: Run Evidence Capture	Pattern: Runtime Skill-BOM (proposed synthesis)	Links artefact identities, participation, decisions, actions, and observations within a run.
Evidence & Feedback: Use Verification (Optional)	Pattern: Verifiable Skill Contract	Assesses run evidence against artefact-associated or configured skill criteria and records verification status.
Evidence & Feedback: Repair & Evolution (Optional)	Patterns: Runtime Skill Repair; Skill–Agent Co-Evolution Loop	Routes bounded run-local repair to Mediation and candidate artefact revisions to Distribution for acceptance.
Policy/Configuration Substrate	Synthesis of documented eligibility conditions, invocation policies, permission rules, and repair bounds	Coordinates rules and configuration consulted across responsibilities.
Identifier Substrate	Synthesis of documented content hashes, version and provenance records, and invocation identifiers; identifier requirements of Runtime Skill-BOM	Supplies shared references to artefacts, versions, runs, decisions, actions, observations, and changes.

Pattern: catalogue pattern allocated to the block. Enabling function: function that pattern solutions presuppose. Optional: selected by requirements (Section 5.5).

The *Identifier Substrate* joins records about artefacts, versions, runs, participation, actions, checks, and changes. Shared references let evidence capture, verification, and feedback retrieve related records across mechanisms. The required granularity follows the question: identifying an invoked skill differs from reconstructing the revision and permission decision governing an action.

5.5. Use and Variation

To apply the RA, an architect identifies the task requirements, allocates documented host mechanisms to responsibilities, and follows their inputs, outputs, and evidence links.

Consider an audit requirement to explain why a commit command was permitted. Claude Code's documented `commit` skill provides a concrete allocation: its `SKILL.md` is the stored artefact; invocation and content loading realise Mediation; and the host permission mechanism realises authorisation and action control [19]. Following this interaction through the RA identifies the evidence needed: the artefact revision, invocation, applicable permission inputs, and action-time decision. The loading record shows which skill participated, whereas the action-time decision record explains why the command was permitted. For this audit requirement, the design choice is to link action-level permission evidence to participation records, with the additional recording and retention cost this entails.

Variation points concern both the responsibilities required and their realisation. Resolution can apply source precedence, scoped visibility, or both; eligibility can rest on validation, trust review, enablement, or environmental conditions; and resources can be bundled, referenced lazily, or generated by scripts. Explicit invocation gives users

direct activation control; descriptor-based selection supports agent-proposed activation; configured preloading supplies named skills or managed instructions loaded in every run. An explicit Skill-BOM makes a run record directly available, while reconstruction joins retained events when needed and depends on sufficient identifiers and event coverage. Use Verification applies when the system must check skill-scoped criteria. Run-local repair applies when bounded adaptation within a run is permitted; persistent evolution applies when reusable artefacts can be revised. A concrete architecture selects these options while preserving the relevant authority, evidence, and change dependencies.

6. Reference Architecture Evaluation: Mapping and Application

Following Step 6 (Section 3.2), we evaluate the RA's correspondence with documented mechanisms through cross-system mapping and its application to concrete requirements through focused technical applications. Galster and Avgeriou discuss mapping and prototyping as evaluation approaches and, for RAs grounded in preceding architectures, emphasise instantiation support [13].

6.1. Evaluation Design

The mapping retains eight systems used in RA construction: Claude Code, OpenAI Codex, GitHub Copilot CLI, Hermes Agent, Aeon, OpenClaw, Microsoft Agent Framework, and Snowflake Cortex Agents. We selected them purposively for maximum variation in ecosystem position (coding agents, open-source runtimes, feedback-driven skill systems, enterprise frameworks, and cloud platforms), evidence depth, and responsibility profile, prioritising first-party detail and non-redundant responsibility coverage while avoiding close-lineage variants. Mapping evidence was assessed in October 2026 using first-party documentation and implementation evidence, including documentation beyond the derivation records.

R denotes documented realisation of the architectural purpose stated in Table 3; P denotes partial support, with part of that purpose or a connection unestablished; N denotes a responsibility not established in the reviewed evidence. Shared harness mechanisms and dedicated skill mechanisms are rated by the same functional rules, with implementation location reported in Section 6.2. A superscript c marks realisation conditional on a particular channel, optional workflow, or non-default configuration. Distribution concerns deliberate adoption or updating of supplied artefacts. Persistent evolution additionally requires a documented workflow connecting operational or evaluation-run feedback, a candidate revision, review or evaluation for its intended use, and adoption. Evidence for the policy and identifier substrates is assessed within the mapped responsibilities and interactions. The first author rated each system against these rules, recording source locators and a rationale for each cell.

The applications examine execution control in Deep Agents and revision acceptance in Hermes. We selected applications with a documented engineering concern, inspectable mechanisms for the relevant responsibilities, observable success and failure conditions, and a local test path. Deep Agents, LangChain's open-source agent harness, extends the assessment beyond the review corpus. Each protocol stated requirements, RA allocations, and expected observations before its recorded checks. The RA supplied the allocations, state distinctions, and checks; documentation and source inspection identified the host mechanisms; execution established their behaviour. In Deep Agents, a deterministic model client fixed the skill read and proposed action; in Hermes, the test driver submitted candidate revisions through the native `skill_manage` tool. Scripted approval or rejection isolated the controls. The checks ran unmodified Deep Agents SDK 0.7.25 and Hermes at pinned commits [48, 49] on disposable fixtures, comparing expected observations with native tool results, prompt contents, and resulting file or skill states. Exploratory diagnostics, run before the protocol was fixed, informed the final Deep Agents configuration and expected observations; two exploratory follow-up Hermes checks compared previews and inspected native change records.

6.2. Cross-System Mapping

Across Table 4, the first six responsibilities are realised in 47 of 48 cells; the 11 R^c ratings depend on reviewed, pinned, or interactively confirmed supply channels for Distribution and on configured approval checks or an enabled sandbox for authorisation and control. The remaining P concerns Distribution in Microsoft Agent Framework, rated for the plain framework agent rather than its Harness Agent variant: its source loading provides no adoption or update channel. Representation & Storage and Activation & Context Realisation used skill mechanisms in all eight systems, and Resolution & Admission used skill mechanisms alone in seven and combined with shared access controls in Snowflake. Runtime Action Control used shared harness mechanisms throughout; Capability Authorisation and Run Evidence Capture used shared harness mechanisms alone or combined with skill mechanisms.

Within Capability Authorisation, Claude Code, Copilot CLI, and Aeon recognise skill declarations; Hermes registers skill-declared environment variables for controlled passthrough to sandboxes, and OpenClaw can allowlist skill-referenced executables on execution nodes. Codex, Microsoft Agent Framework, and Snowflake establish host or resource controls without a documented skill-declaration input.

The clearest variation concerns evidence, verification, and acceptance of change. Claude Code and Codex can link skill participation, permission decisions, and tool results through shared prompt or turn identifiers in configured telemetry; Copilot CLI retains related session events. These joins realise Run Evidence Capture. Revision-level reconstruction, as a Skill-BOM requires, needs more: the cited Claude Code and Codex event schemas identify skills without their artefact revision, whereas Copilot retains loaded

Table 4
Documented responsibility realisations in the selected systems.

System	Supply Chain		Mediation		Execution Control		Evidence & Feedback		
	Dist.	Rep.	Res./Ad.	Act./Ctx.	Auth.	Ctrl.	Evid.	Verif.	Persist. evol.
Claude Code	R^c	R	R	R	R	R	R^c	R^c	R^c
OpenAI Codex	R^c	R	R	R	R	R	R^c	R^c	R^c
GitHub Copilot CLI	R^c	R	R	R	R	R	R	N	R^c
Hermes Agent	R^c	R	R	R	R	R	P	N	R^c
Aeon	R	R	R	R	R	R^c	P	P	R^c
OpenClaw	R^c	R	R	R	R^c	R^c	P	N	P
Microsoft Agent Framework	P	R	R	R	R^c	R^c	P	P	N
Snowflake Cortex Agents	R^c	R	R	R	R	R	P	P	N

Dist.: Distribution; Rep.: Representation & Storage; Res./Ad.: Resolution & Admission; Act./Ctx.: Activation & Context Realisation; Auth.: Capability Authorisation; Ctrl.: Runtime Action Control; Evid.: Run Evidence Capture; Verif.: Use Verification; Persist. evol.: persistent-evolution branch of Repair & Evolution (run-local repair was not established for any system).
R: realised; R^c: realised under a stated condition; P: partial; N: not established in the reviewed evidence.

skill content. The remaining five Run Evidence Capture ratings are P because the reviewed records leave part of the participation–decision–observation connection unestablished.

The Use Verification ratings separate skill-specific criteria from general evaluation facilities. Claude Code's optional skill/plugin evaluation workflow applies skill-targeted graders to dedicated evaluation runs [19]. OpenAI's documented, optional Codex evaluation workflow runs a skill-specific prompt set, checks the retained run trace with scripts, and saves a structured rubric assessment [50]. Aeon, Microsoft Agent Framework, and Snowflake provide relevant evaluation facilities, but the reviewed paths do not establish the complete skill-specific criterion-to-observation link.

Persistent evolution varies in how candidate revisions enter reusable supply. Claude Code's evaluation loop qualifies when a candidate revised using evaluation-run feedback enters supply through a controlled release boundary. Codex qualifies when its skill-creator iteration and evaluation workflow feeds a reviewed plugin release [50]. Copilot's `/chronicle improve` qualifies through proposed changes to managed repository instructions and user selection before writing (Section 2.2). With skill-write approval enabled, Hermes qualifies through its documented review path for targeted replacements; the applications below also examine full rewrites. Aeon qualifies through enabled feedback-led pull requests reviewed before merge. OpenClaw's Skill Workshop supports feedback-driven changes with write-time validation, version history, and undo; without review before reuse, it gives partial support.

6.3. Focused Applications

Table 5 groups the eight Deep Agents and six Hermes checks by RA decision.

Deep Agents: participation and execution control. A report skill's bundled helper sums the input values 1, 2, and 3; the same helper can also overwrite a separate protected file, which must remain unchanged. The configuration combines a shell-capable default backend with a `/protected/` filesystem route and a write-deny rule for `/protected/**`. Deep Agents documents that this rule applies to its built-in file tools; `execute` reaches the same file through the shell backend. Because the RA routes executable operations through Runtime Action Control, we checked both routes and set `interrupt_on` for `execute`, requiring an approval decision for each call to that tool. With the skill loaded, rejection prevented the modifying command; approval of the report command produced `total=6` and preserved the protected file.

The declaration comparison examines how skill metadata relates to host permissions. With identical skill body, helper, inputs, and proposed commands, both `allowed-tools: execute` and `allowed-tools: read_file` appeared in the prompt. Without the approval gate, the shell modified the protected file under either declaration; with it, `allowed-tools: execute` did not suppress the approval request. This matches the skill middleware, which renders `allowed-tools` in the prompt without applying it to permissions.

Hermes: approval and revision acceptance. With `skills.write_approval=true`, proposed changes entered a pending store separate from the current artefact, and approved changes had to pass structural validation before replacing it. Approval of a valid revision made its exact content readable; approval of malformed content failed structural validation and preserved the current version. This exercises the RA path from candidate revision through Distribution's acceptance to supply: an approval decision and successful acceptance are distinct states.

Checking the preview shown to the approver exposed a preview–apply mismatch for the advertised full-rewrite operation. The preview failed in the same way for valid and malformed candidates, yet approving a valid candidate applied its content: the preview parses the full-content `patch` as a search-and-replace edit with an empty search string, whereas approval applies it as a full rewrite. Hermes's preview code states that the preview should match what approval applies. The native mutation ledger retained

Table 5
RA decisions, tested conditions, criteria, and observations in the focused applications.

RA decision	Tested condition	Criterion	Observation
Deep Agents: protect a file while keeping the report skill usable			
Runtime Action Control: file-tool denial and shell approval	File-tool edit vs. ungated shell helper, both targeting the protected file	Protected file unchanged on each route	Met for the file tool; not met for the ungated shell, which modified the file
	Gated shell: reject the modifying command; approve the report command	File unchanged; report produced	Met: rejection blocked the change; approval produced the report with the file unchanged
Capability Authorisation: distinguish declarations from host permissions	`allowed-tools: execute` vs. `read_file` vs. no declaration, ungated and gated with rejection	Same enforcement outcomes with and without declarations	Met: both declarations reached the prompt without changing enforcement
Hermes: accept persistent revisions			
Distribution: separate candidate from current artefact	Stage a valid candidate, then reject it	Current skill unchanged	Met: current skill unchanged while pending and after rejection
Distribution: separate approval, validation, and adoption	Approve a valid vs. a malformed candidate	Valid revision applied; malformed revision blocked by validation	Met: valid revision applied and readable; malformed content failed validation, current version preserved
Distribution: preview before acceptance	Preview valid and malformed full-rewrite candidates[a]	Preview shows each candidate	Not met: identical failed previews, yet valid content was applied on approval
Identifier Substrate: link candidate, decision, and revision	Inspect native records after acceptance[a]	Records link all three	Not met: native ledger recorded revision hashes without candidate or decision links

Gated: each `execute` (shell) call requires approval through `interrupt_on`. [a]Includes exploratory follow-up checks (Section 6.1).

before/after revision hashes but no link to the candidate or approval decision; the test driver assembled that association.

6.4. Findings and Discussion

Evidence and feedback need explicit allocation. Evidence & Feedback holds 14 of the 15 P and N ratings in Table 4, 11 of them in Run Evidence Capture and Use Verification, whose patterns have mainly research-informed support; run-local repair was not established in any mapping. Only one of the layer's 24 ratings is an unconditional R, against 36 of the 48 in the other layers, and 20 of all 57 realised ratings depend on a channel, workflow, or configuration. Requirements for run-specific provenance, skill-scoped verification, or governed revision therefore need explicitly allocated and configured mechanisms.

A skill's permission effects depend on host semantics and configuration. The same `allowed-tools` field pre-approves tools in Claude Code and GitHub Copilot CLI (Section 4.2) but left enforcement unchanged in the tested Deep Agents configuration. Pre-approval alone does not restrict the tool set [19], and OpenClaw's gateway and node defaults let skill helpers run without approval prompts. Adopting a skill therefore requires checking how the target host interprets its declarations and which controls the deployed configuration enables.

Review should follow interactions across responsibilities. Both applications allocated their requirements to existing RA responsibilities, and both separated documented controls from their effective coverage or operation: the Deep Agents file rule left the shell route open, and the Hermes full-rewrite preview did not match what approval applied. Architecture review should therefore follow each route to a protected resource and each candidate through preview, approval, validation, and later supply, with the Identifier Substrate linking the candidate, decision, and resulting revision.

7. Threats to Validity

We discuss construct, internal, and external validity and reliability [51].

7.1. Construct Validity

Translating vendor-specific descriptions into the skill definitions and mapping rules involves judgement, particularly where shared harness mechanisms implement skill-related functions. Public sources also differ in how much they document about controls, execution records, and feedback, and 50 of the 51 academic papers were assessed in their arXiv versions, 43 of them from 2026. Both affect

the visibility of mechanisms and the strength of support for individual patterns. Mapping ratings describe documented realisation; the focused applications test operation.

7.2. Internal Validity

The authors developed and applied the RA, and the eight mapped systems belong to the review corpus, which can favour interpretations consistent with the proposed structure. The mapping rules registered 15 partial or unestablished realisations, and Deep Agents tests the RA outside the corpus. Fixed action proposals, candidate revisions, and approval decisions attribute each application outcome to host enforcement or acceptance, independently of model behaviour.

7.3. External Validity

Rating frequencies describe the eight purposively selected systems. Searches used skill terminology, and academic searches were restricted to titles from 2025 onward; earlier work and equivalent mechanisms described as instructions or rules can therefore be underrepresented.

Host policies, interfaces, and documentation evolve, so mechanism-level findings attach to the documented or tested versions and configurations. The patterns abstract design decisions from vendor-specific mechanisms, and the RA compares implementations at the level of responsibilities; new kinds of mechanism may require revised pattern solutions or responsibility boundaries.

7.4. Reliability

After the jointly assessed pilot, the first author performed the remaining source assessment, synthesis, and mapping; records linking source practices to patterns, responsibilities, and ratings make these interpretive decisions inspectable.

Web search results change over time; the recorded queries, follow-up targets, and source records define the reviewed corpus. Replays of the technical applications with fixed inputs and pinned implementations reproduced the recorded outcomes, including those of the exploratory checks.

8. Related Work

8.1. Agent Harness and Harness Engineering

Agent harness research examines how context, tools, execution, and observation support model-driven action. SWE-agent studies the agent–computer interface, and OpenHands provides sandboxed execution and a shared event stream [52, 53]. Tang et al. survey harness engineering across workflows, memory, skill libraries, and multi-agent orchestration, relating harness design to model capabilities and system performance [54]. Natural-Language Agent Harnesses expresses harness control logic in editable documents that a shared runtime interprets into calls, gates, and contracts [55], and Agentic Harness Engineering evolves the harness around a fixed model from observations of components, trajectories, and decisions [56]. Both make the harness itself editable and evolvable; our patterns govern the participation, effects, and revision of the skill artefacts that a harness supplies.

Architectural work identifies responsibilities that our RA builds on. Lu et al.'s reference architecture for foundation-model agents includes guardrails, a black-box recorder, and an AI bill of materials for provenance [8]. Liu et al.'s agent pattern catalogue documents forces, trade-offs, and known uses; its registry, adapter, and evaluator patterns cover selection, tool integration, and assessment, with Voyager's skill library among the registry examples [9]. Our catalogue specialises these concerns for identifiable skill artefacts and their staged participation, and the proposed Runtime Skill-BOM combines the provenance and recording roles for skills, linking artefact revisions, participation events, applicable permission decisions, and execution evidence in a per-run record. AgentArcEval complements such architectures with scenario-based evaluation [57], applicable to concrete architectures instantiated from the RA.

8.2. Agent Skills and Skill Harnessing

Skill research contributes mechanisms, surveys, and empirical evidence. Systems build reusable program and workflow libraries [58, 59] and verify and select evolving skills [60, 61]; surveys organise skill representation, acquisition, retrieval, and evolution [62]. SkillsBench and realistic-usage evaluations examine skill utility across tasks and deployment conditions, counterfactual trace auditing examines how skills affect trajectories [63, 64, 65], and security studies document malicious skills, skill-file injection, and credential leakage [66, 67, 68]. The RA assigns the controls and evidence these findings call for to admission, permission determination, runtime enforcement, evidence capture, and use verification.

Several accounts develop the architectural relationships between skills and host controls. Zhou et al.'s externalisation review connects skills to memory, protocols, and the harness through selection, permission-checked binding, approval gates, audit records, and lifecycle feedback [2]. Xu and Yan describe staged loading and execution-context changes, including tool pre-approval, and propose verification gates and trust tiers for lifecycle governance [10]. SkillGuard treats skills as permission-bearing artefacts and combines manifests, runtime permission control, and policy enforcement over their context influence and side effects [69]. These accounts establish the coupling between skill participation and host controls. Our RA allocates it to two responsibilities: Capability Authorisation interprets recognised skill declarations within host policy, and Runtime Action Control enforces the resulting decisions on executable operations (Section 6.3).

Badhe et al. propose a reference architecture organised around nine skill-lifecycle stages, from discovery to security governance, covering permission gating, verification, repair, and both temporary adaptation and persistent evolution [11]. Our RA organises decisions by the objects they govern and specifies the operational and evidence dependencies between responsibilities: commands initiated during resource materialisation pass through Runtime Action Control, and persistent revisions return to Distribution for acceptance.

Jiang et al. catalogue seven skill design patterns, each characterised by representation, representative systems, strengths, and risks [70]. Their progressive-disclosure and self-evolving-library patterns overlap Progressive Skill Activation and Skill–Agent Co-Evolution Loop, marketplace distribution overlaps Skill Extension Bundle and the RA's Distribution responsibility, and their proposed trust tiers parallel Skill–Execution Authority Separation. Our patterns are derived from documented practices, state forces and consequences, and each has a principal allocation in the RA.

9. Conclusion

This paper presents ten architectural patterns for agent skill harnessing, derived from a multivocal literature review, and a reference architecture that connects decisions about a skill across supply, runtime participation, execution control, evidence, and change. Mappings of eight systems show supply, mediation, and execution control broadly realised, with 14 of the 15 partial or unestablished ratings in evidence, verification, and persistent evolution; where these responsibilities are required, they need explicitly allocated mechanisms. Applying the architecture in Deep Agents and Hermes turned requirements into checks: in Deep Agents, an approval gate closed a shell route that file-tool rules left open; in Hermes, approval and validated acceptance were separate, but the full-rewrite preview did not match the applied revision. The architecture therefore directs review to how hosts interpret skill declarations, which controls cover each executable route, and which records link a revision to its candidate and approval.

Acknowledgement

Funding: This work was supported by the Responsible AI Research (RAIR) Centre, Adelaide, Australia.

Declaration of Generative AI Use

During the preparation and revision of this manuscript, the authors used *ChatGPT (OpenAI)* and *Claude (Anthropic)* to assist with writing tasks such as language polishing, tightening of phrasing, and consistency checking. After using these tools, the authors carefully reviewed, revised, and edited the text, and take full responsibility for all content of the published article.

CRediT authorship contribution statement

Boming Xia: Conceptualization, Methodology, Writing - Original draft, Writing - Review & Editing. **Liming Zhu:** Supervision, Conceptualization, Writing - Review & Editing. **Zhenchang Xing:** Supervision, Conceptualization, Writing - Review & Editing. **Qinghua Lu:** Supervision, Conceptualization, Writing - Review & Editing. **Dino Sejdinovic:** Supervision, Writing - Review & Editing. **Xiwei Xu:** Writing - Review & Editing.

References

[1] Anthropic, Building effective agents, 2024. URL: https://www.anthropic.com/research/building-effective-agents.

[2] C. Zhou, H. Chai, W. Chen, Z. Guo, R. Shan, Y. Song, T. Xu, Y. Yang, A. Yu, W. Zhang, et al., Externalization in LLM agents: A unified review of memory, skills, protocols and harness engineering, arXiv preprint arXiv:2604.08224 (2026).

[3] Anthropic, Equipping agents for the real world with agent skills, https://www.anthropic.com/research/equipping-agents-for-the-real-world-with-agent-skills, 2025. Published 16 October 2025; accessed 7 May 2026.

[4] Agent Skills, Agent skills overview, Agent Skills Documentation, n.d. URL: https://agentskills.io/home, accessed 7 May 2026.

[5] H. Liu, Y. Ming, S. Joty, C. Zhao, Harnessing LLM agents with skill programs, arXiv preprint arXiv:2605.17734 (2026).

[6] Agent Skills, Agent skills specification, n.d. URL: https://agentskills.io/specification, living specification. Accessed 8 October 2026.

[7] GitHub, Adding agent skills for GitHub Copilot CLI, n.d. URL: https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/add-skills, documentation. Accessed 8 October 2026.

[8] Q. Lu, L. Zhu, X. Xu, Z. Xing, S. Harrer, J. Whittle, Towards responsible generative AI: A reference architecture for designing foundation model based agents, in: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), 2024, pp. 119–126. doi:10.1109/ICSA-C63560.2024.00028.

[9] Y. Liu, S. K. Lo, Q. Lu, L. Zhu, D. Zhao, X. Xu, S. Harrer, J. Whittle, Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents, Journal of Systems and Software 220 (2025) 112278.

[10] R. Xu, Y. Yan, Agent skills for large language models: Architecture, acquisition, security, and the path forward, arXiv preprint arXiv:2602.12430v4, 2026.

[11] S. Badhe, D. Shah, P. Tiwari, N. Kathrotia, A systematic survey of agentic skills: Architecture, lifecycle, and security, arXiv preprint arXiv:2608.29596v2, 2026.

[12] V. Garousi, M. Felderer, M. V. Mäntylä, Guidelines for including grey literature and conducting multivocal literature reviews in software engineering, Information and Software Technology 106 (2019) 101–121.

[13] M. Galster, P. Avgeriou, Empirically-grounded reference architectures: a proposal, in: Proceedings of the joint ACM SIGSOFT conference–QoSA and ACM SIGSOFT symposium–ISARCS on Quality of software architectures–QoSA and architecting critical systems–ISARCS, 2011, pp. 153–158.

[14] Model Context Protocol, Model Context Protocol specification, version 2026-07-28, 2026. URL: https://modelcontextprotocol.io/specification/2026-07-28, accessed 22 October 2026.

[15] Model Context Protocol, Skills extension, n.d. URL: https://modelcontextprotocol.io/extensions/skills/overview, accessed 22 October 2026.

[16] Anthropic, PDF skill: Instructions and form-processing resources, n.d. URL: https://github.com/anthropics/skills/tree/main/skills/pdf, public skill package, including forms.md and helper scripts. Accessed 9 October 2026.

[17] Anthropic, Create custom subagents, n.d. URL: https://code.claude.com/docs/en/sub-agents#preload-skills-into-subagents, Claude Code documentation, including skill preloading. Accessed 9 October 2026.

[18] OpenHands, Agent skills & context, n.d. URL: https://docs.openhands.dev/sdk/guides/skill, SDK documentation. Accessed 8 October 2026.

[19] Anthropic, Extend Claude with skills, 2026. URL: https://code.claude.com/docs/en/skills, Claude Code documentation. Accessed 8 October 2026.

[20] B. Kitchenham, S. Charters, Guidelines for Performing Systematic Literature Reviews in Software Engineering, Technical Report EBSE-2007-01, Keele University and Durham University, 2007.

[21] S. Angelov, P. Grefen, D. Greefhorst, A classification of software reference architectures: Analyzing their success and effectiveness, in: 2009 Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture, IEEE, 2009, pp. 141–150.

[22] ISO/IEC/IEEE, ISO/IEC/IEEE 42010:2022: Software, systems and enterprise – Architecture description, 2022. URL: https://www.iso.org/standard/74393.html, published 2022-11.

[23] Perplexity, Designing, refining, and maintaining agent skills at Perplexity, Perplexity Research, 2026. URL: https://www.perplexity.ai/hub/blog/designing-refining-and-maintaining-agent-skills-at-perplexity.

[24] A. Metere, Skills as Verifiable Artifacts: A Trust Schema and a Biconditional Correctness Criterion for Human-in-the-Loop Agent Runtimes, 2026. doi:10.48550/ARXIV.2605.00424.

[25] T. Chen, Y. Li, M. Solodko, S. Wang, N. Jiang, T. Cui, J. Hao, J. Ko, S. Abdali, L. Xu, et al., CUA-Skill: Develop skills for computer using agent, arXiv preprint arXiv:2601.21123 (2026).

[26] Y. Ouyang, Y. Xiao, Y. Gu, X. Zhang, SkCC: Portable and secure skill compilation for cross-framework LLM agents, arXiv preprint arXiv:2605.03353 (2026).

[27] Z. Lu, Y. Zuo, Y. Nie, X. He, W. Fan, L. Qi, S. Jin, ContractSkill: Repairable contract-based skills for multimodal web agents, arXiv preprint arXiv:2603.20340 (2026).

[28] Z. Z. Wang, A. Gandhi, G. Neubig, D. Fried, Inducing programmatic skills for agentic tasks, in: Conference on Language Modeling (COLM), 2025.

[29] Z. Ren, L. Yue, R. Li, Y. Wang, S. Xu, H. Liu, S. Pan, S. Di, FlowEvo: Self-evolving agents through the co-evolution of workflows and executable skills, in: Conference on Language Modeling (COLM), 2026. URL: https://arxiv.org/abs/2607.21596.

[30] B. Xia, T. Bi, Z. Xing, Q. Lu, L. Zhu, An empirical study on software bill of materials: Where we stand and the road ahead, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 2630–2642.

[31] Y. Liang, R. Zhong, H. Xu, C. Jiang, Y. Zhong, R. Fang, J.-C. Gu, S. Deng, Y. Yao, M. Wang, et al., SkillNet: Create, evaluate, and connect AI skills, arXiv preprint arXiv:2603.04448 (2026).

[32] Q. Liang, H. Wang, Z. Liang, Y. Liu, From skill text to skill structure: The scheduling-structural-logical representation for agent skills, arXiv preprint arXiv:2604.24026 (2026).

[33] V. P. Bhardwaj, Formal analysis and supply chain security for agentic AI skills, arXiv preprint arXiv:2603.00195v1 (2026).

[34] Y. Peng, Y. Sun, J. Ding, B. Yan, Z. Wang, C. Wang, C. Shu, J.-G. Zhou, S. Wang, oxo-call: Documentation-grounded skill augmentation for accurate bioinformatics command-line generation with large language models, arXiv preprint arXiv:2604.12387 (2026).

[35] T. Shen, Y. Feng, K. Zhu, X. Jia, Y. Liu, L. Zhang, Sealing the audit-runtime gap for LLM skills, arXiv preprint arXiv:2605.05274 (2026).

[36] H. Liu, H. Yang, T. Jiang, B. Tang, F. Xiong, Y. Luo, Z. Li, SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution, arXiv preprint arXiv:2605.18401 (2026).

[37] K. Huang, J. Huang, Audited skill-graph self-improvement for agentic LLMs via verifiable rewards, experience synthesis, and continual memory, arXiv preprint arXiv:2512.23760 (2025).

[38] G. Zhao, Q. Shi, X. Xiao, X. Zhang, T. Yang, L. Sun, Thinking with reasoning skills: Fewer tokens, more accuracy, in: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track), 2026, pp. 2295–2308. doi:10.18653/v1/2026.acl-industry.154.

[39] Y. Shi, Y. Chen, Z. Lu, Y. Miao, S. Liu, Q. Gu, X. Cai, X. Wang, A. Zhang, Skill1: Unified evolution of skill-augmented agents via reinforcement learning, arXiv preprint arXiv:2605.06130 (2026).

[40] G. Jiang, Z. Su, X. Qu, Y. R. Fung, XSkill: Continual learning from experience and skills in multimodal agents, in: Proceedings of the 43rd International Conference on Machine Learning (ICML), volume 306 of *Proceedings of Machine Learning Research*, 2026, pp. 53078–53104. URL: https://proceedings.mlr.press/v306/jiang26aq.html.

[41] B. Zheng, M. Y. Fatemi, X. Jin, Z. Z. Wang, A. Gandhi, Y. Song, Y. Gu, J. Srinivasa, G. Liu, G. Neubig, et al., SkillWeaver: Web agents can self-improve by discovering and honing skills, arXiv preprint arXiv:2504.07079 (2025).

[42] J. Ni, Y. Liu, X. Liu, Y. Sun, M. Zhou, P. Cheng, D. Wang, E. Zhao, X. Jiang, G. Jiang, Trace2Skill: Distill trajectory-local lessons into transferable agent skills, arXiv preprint arXiv:2603.25158 (2026).

[43] S. Ouyang, J. Yan, Y. Chen, R. Han, Z. Wang, B. D. Mishra, R. Meng, C.-L. Li, Y. Jiao, K. Zha, et al., SkillOS: Learning skill curation for self-evolving agents, arXiv preprint arXiv:2605.06614 (2026).

[44] X. Wu, Z. Li, G. Shi, A. Duffy, T. Marques, M. L. Olson, T. Zhou, D. Manocha, Co-Evolving LLM Decision and Skill Bank Agents for Long-Horizon Tasks, 2026. doi:10.48550/arXiv.2604.20987.

[45] Z. Ma, S. Yang, Y. Ji, X. Wang, Y. Wang, Y. Hu, T. Huang, X. Chu, SkillClaw: Let skills evolve collectively with agentic evolver, arXiv preprint arXiv:2604.08377 (2026).

[46] T. Xia, L. Hu, Y. Sun, M. Xu, L. Xu, S. Wang, W. Xu, J. Jiang, GraSP: Graph-structured skill compositions for LLM agents, arXiv preprint arXiv:2604.17870 (2026).

[47] Y. Li, Y. Dou, J.-J. Shao, Y. Lyu, I. Tsang, H. Yin, SkillTracer: Structural failure attribution and refinement of agentic skills in long-horizon web tasks, in: Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2, 2026, pp. 2897–2908. doi:10.1145/3770855.3817981.

[48] LangChain, Deep Agents: Source revision used for the reference architecture application, 2026. URL: https://github.com/langchain-ai/deepagents/tree/c6c340d47d2013a2e7b7230698f73977b7de6756, SDK version 0.7.25. Accessed 14 October 2026.

[49] Nous Research, Hermes Agent: Source revision used for the reference architecture application, 2026. URL: https://github.com/NousResearch/hermes-agent/tree/d97007c32d6340f7ba79ac1dfb5773cc00846448, accessed 14 October 2026.

[50] D. Kundel, G. Chua, Testing agent skills systematically with evals, 2026. URL: https://developers.openai.com/blog/eval-skills, openAI Developers. Published 22 January 2026. Accessed 23 October 2026.

[51] P. Runeson, M. Höst, Guidelines for conducting and reporting case study research in software engineering, Empirical Software Engineering 14 (2009) 131–164.

[52] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, SWE-agent: Agent-computer interfaces enable automated software engineering, in: Advances in Neural Information Processing Systems, volume 37, 2024, pp. 50528–50652.

[53] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al., OpenHands: An open platform for AI software developers as generalist agents, in: International Conference on Learning Representations, volume 2025, 2025, pp. 65882–65919.

[54] X. Tang, H. Peng, G. Chen, Y. Shi, Z. Su, P. Liu, W. X. Zhao, Y. Li, Z. Xue, Agent systems with harness engineering: A systematic survey, Frontiers of Computer Science (2026). In press. 10.1007/s11704-026-61123-6.

[55] L. Pan, L. Zou, S. Guo, J. Ni, H.-T. Zheng, Natural-language agent harnesses, arXiv preprint arXiv:2603.25723 (2026).

[56] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, Y.-G. Jiang, Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, arXiv preprint arXiv:2604.25850 (2026).

[57] Q. Lu, D. Zhao, Y. Liu, H. Zhang, L. Zhu, X. Xu, A. Shi, T. Tan, R. Kazman, AgentArcEval: An architecture evaluation method for foundation model based agents, Journal of Systems and Software 232 (2026) 112656.

[58] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar, Voyager: An open-ended embodied agent with large language models, Trans. Mach. Learn. Res. 2024 (2024).

[59] Z. Z. Wang, J. Mao, D. Fried, G. Neubig, Agent workflow memory, in: Forty-second International Conference on Machine Learning, volume 267 of *Proceedings of Machine Learning Research*, PMLR, 2025, pp. 63897–63911. URL: https://proceedings.mlr.press/v267/wang25bx.

html.

[60] H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng, et al., CoEvoSkills: Self-evolving agent skills via co-evolutionary verification, in: Conference on Language Modeling (COLM), 2026.

[61] S. Alzubi, N. Provenzano, J. Bingham, W. Chen, T. Vu, EvoSkill: Automated skill discovery for multi-agent systems, arXiv preprint arXiv:2603.02766 (2026).

[62] Y. Zhou, W. Shu, Y. Su, W. Du, Y. Fang, X. Lin, A comprehensive survey on agent skills: Taxonomy, techniques, and applications, arXiv preprint arXiv:2605.07358 (2026).

[63] X. Li, Y. Liu, W. Chen, B. You, Z. Di, Y. He, S. Zheng, K. W. Choe, J. Sun, S. Wang, et al., SkillsBench: Benchmarking how well agent skills work across diverse tasks, arXiv preprint arXiv:2602.12670 (2026).

[64] Y. Liu, J. Ji, L. An, T. Jaakkola, Y. Zhang, S. Chang, How well do agentic skills work in the wild: Benchmarking LLM skill usage in realistic settings, in: Conference on Language Modeling (COLM), 2026.

[65] X. Zhou, J. Liu, L. Li, R. A. Rossi, X. Hu, Counterfactual trace auditing of LLM agent skills, arXiv preprint arXiv:2605.11946 (2026).

[66] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, L. Y. Zhang, "Do Not Mention This to the User": Detecting and understanding malicious agent skills in the wild, in: 35th USENIX Security Symposium (USENIX Security 26), 2026, pp. 1727–1746.

[67] D. Schmotz, L. Beurer-Kellner, S. Abdelnabi, M. Andriushchenko, Skill-Inject: Measuring agent vulnerability to skill file attacks, arXiv preprint arXiv:2602.20156 (2026).

[68] Z. Chen, Y. Zhang, Y. Liu, G. Deng, Y. Li, Y. Zhang, J. Ning, L. Y. Zhang, L. Ma, Z. Li, How your credentials are leaked by LLM agent skills: An empirical study, in: Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE), 2026, pp. 536–548. doi:10.1145/3832783.3834373.

[69] S. Pan, X. Sun, T. Zhang, D. Liao, K. Yang, Z. Xing, SkillGuard: A permission-centric framework for agent skill security, arXiv preprint arXiv:2606.03024 (2026).

[70] Y. Jiang, D. Li, H. Deng, B. Ma, X. Wang, Q. Wang, G. Yu, SoK: Agentic skills—beyond tool use in LLM agents, arXiv preprint arXiv:2602.20867, 2026.